

Práctica 1

Analizador Léxico - Lenguaje Lua

Profesor: Felipe Restrepo Calle (ferestrepoca@unal.edu.co)

Monitor: Reinaldo Toledo Leguizamón (rtoledol@unal.edu.co)

Dado el código fuente de un programa en el lenguaje de programación Lua, su tarea consiste en realizar el **análisis léxico** sobre dicho código de acuerdo a las especificaciones descritas a continuación. Para realizar la implementación se podrán utilizar únicamente los siguientes lenguajes de programación: **Python 3.9, C/C++ y Java**.

Entrada

La **entrada** consiste en el código fuente de un programa en **Lua**, el cual puede estar correcto a nivel léxico o no. Es decir, su programa debe recibir por la **entrada estándar (consola)** el código fuente de un programa escrito en el lenguaje de programación **Lua**.

Para evaluar el analizador léxico de forma automática, su programa debe realizar el análisis léxico de la entrada dada y generar la salida adecuada, de acuerdo a las especificaciones dadas a continuación.

Salida

Las salidas se deben generar por la **salida estándar (consola)**. Note que en estas instancias no interesa si el programa tiene errores de sintaxis o errores semánticos porque, por el momento, nos enfocaremos solamente en el análisis léxico.

La **salida** consiste en una **lista de tokens** (uno por línea) especificados en el siguiente formato:

<tipo_de_token,lexema,fila,columna>

En el caso de las **palabras reservadas**, el tipo de token y el lexema son iguales. Entonces, solo se imprimirá en el siguiente formato:

<palabra_reservada, fila, columna>

Por ejemplo, a la hora de imprimir la palabra reservada "global", debe imprimir la misma palabra reservada en el espacio de palabra_reservada, así: <global, fila, columna>, donde fila y columna serán los números correspondientes.

A continuación puede encontrar un ejemplo más detallado:

Entrada	Salida
<pre>print false in print end return</pre>	<pre><print,1,1> <false,2,5> <in,3,6> <print,5,2> <end,5,9> <retornar,6,6></pre>

Por otra parte, note que parte de su trabajo consiste en identificar cuáles son todas las palabras reservadas a partir de la definición del lenguaje que se encuentra disponible en el [manual de referencia](#).

No obstante, debe tener en cuenta las siguientes [observaciones](#) para comprender el alcance del lenguaje que se está contemplando para el desarrollo de esta práctica y los próximos trabajos de la asignatura.

El tipo de token correspondiente a los **identificadores** es **id**; estos se deberán imprimir en el siguiente formato:

<id,lexema,fila,columna>

Por ejemplo, el lexema `mi_variable`, al no ser reconocido como palabra reservada, corresponderá a un identificador (**id**). De manera que la salida para el token mencionado sería:

<id,mi_variable,fila,columna>

Tenga en cuenta que un identificador, en este lenguaje, puede ser cualquier cadena de **letras latinas**, **dígitos arábigos** o **guiones bajos**, siempre que no comiencen con un dígito y no sean palabras reservadas.

Recuerde que este lenguaje **es case sensitive**, lo cual quiere decir que se hace distinción entre mayúsculas y minúsculas. Por lo tanto, si en el código fuente aparece la secuencia `PRINT`, el analizador léxico deberá retornar el token como un identificador. Por otro lado, si aparece la secuencia `print`, el analizador léxico tokenizará la palabra reservada correctamente.

Por ejemplo:

Entrada	Salida
<pre>PRINT print Print Mi_Variable wHILE WHILE while</pre>	<pre><id,PRINT,1,1> <print,1,7> <id,Print,3,1> <id,Mi_Variable,3,7> <id,wHILE,6,1> <id,WHILE,6,7> <while,6,13></pre>

Veamos otro ejemplo considerando **comentarios**:

Entrada	Salida
<pre>until repeat in false o else --this is for videogames enjoyers in true Nene -- perfect --[[I don't have enough creativity to write comments]] v</pre>	<pre><until,1,1> <repeat,2,4> <in,2,11> <false,2,14> <id,o,2,20> <else,2,22> <in,5,10> <true,5,13> <id,Nene,5,18> <id,v,10,4></pre>

En el ejemplo anterior se puede evidenciar que los **comentarios** se deben ignorar.

- **NOTA:** Recuerde que, de acuerdo con el manual de referencia, existen dos tipos de comentarios, los que ocupan una sola línea denotados por la secuencia `--`; y los bloques de comentarios que pueden ocupar una o varias líneas denotados por la cláusula de inicio `--[[` y la cláusula de fin `]]`. Si quisiéramos poner un comentario seguido a un token válido del lenguaje, en una misma línea, el analizador deberá ignorar solamente la parte del comentario. Por ejemplo, para la siguiente cadena de entrada:

```
nil --comment
```

El analizador sólo detectará la palabra reservada `nil`, ignorando el comentario en seguida.

A continuación se presenta un ejemplo más completo que nos permitirá aclarar el manejo de otros elementos del lenguaje como **símbolos especiales** y **cadenas no específicas**:

Entrada	Salida
<pre>local edad = 16 if edad>=18 then print("Eres mayor de edad.") end</pre>	<pre><local,1,1> <id,edad,1,7> <tkn_assign,1,12> <tkn_num,16,1,14> <if,3,1> <id,edad,3,4> <tkn_geq,3,8> <tkn_num,18,3,10> <then,3,13> <print,4,3> <tkn_opening_par,4,8> <tkn_str,Eres mayor de edad.,4,9> <tkn_closing_par,4,30> <end,5,1></pre>

A partir de lo anterior se tiene que:

- Para reportar las **cadenas no propias del lenguaje** (valores numéricos y cadenas de caracteres), se debe reportar el **token** correspondiente y, posteriormente, reportar su **lexema** correspondiente. Para los valores numéricos, se empleará el formato `<tkn_tipo_de_dato,lexema,fil,columna>`; es decir que se tendrán los siguientes dos formatos:

`<tkn_num,lexema,fil,columna>`

`<tkn_str,lexema,fil,columna>`

Donde el primero se debe utilizar para reportar valores numéricos (el lexema es el número, aunque puede ser entero o decimal) y el segundo para reportar valores correspondientes a cadenas de caracteres (el lexema es el contenido de la cadena).

Las cadenas de texto correspondientes al tipo de dato **string**, pueden ser denotadas por comillas dobles o comillas simples. Por consiguiente, cuentan como tokens, y se deberán imprimir bajo el siguiente formato:

`<tkn_str,lexema,fil,columna>`

Donde el lexema debe imprimirse sin las comillas (dobles o simples) que delimitan la cadena. También es importante considerar que los lexemas deben mostrar la cadena tal como está en el código fuente, respetando las mayúsculas, minúsculas y la cantidad de espacios, así como las respectivas secuencias de escape que se especifican en el manual de referencia. Sin embargo, y tal y como se evidencia en el ejemplo anterior, es posible especificar comillas simples explícitamente en una cadena de caracteres siempre y cuando se haya delimitado la cadena con comillas dobles. Para el caso contrario, es posible especificar directamente comillas dobles dentro de una cadena de caracteres delimitando la cadena con comillas simples. No obstante, también es posible especificar comillas del mismo tipo de la delimitación haciendo uso de las secuencias de escape.

- Para nombrar los tokens correspondientes a **operadores y símbolos del lenguaje** se utilizará la siguiente convención:

`tkn_nombre_token`

Donde **nombre_token** estará dado a partir de la siguiente tabla:

Símbolo u Operador	nombre_token
&	bit_and
	bit_or
~	bitex_or
>>	right_shift
<<	left_shift

;	semicolon
:	colon
,	comma
.	period
::	goto
..	concat
...	varargs
{	opening_key
}	closing_key
[	opening_bra
]	closing_bra
(	opening_par
)	closing_par
#	length
+	plus
-	minus
*	times
/	div
//	floor_div
^	power
%	mod
==	equal
~=	neq
<=	leq
>=	geq
>	greater
<	less
=	assign

Note que cuando el nombre está compuesto por varias palabras, ellas están separadas por guiones bajos (“_”), es decir que se utiliza la convención de nombrado `snake_case`. Entonces, los operadores y símbolos especiales deben imprimirse en el siguiente formato:

`<tkn_nombre_token, fila, columna>`

En los ejemplos anteriores se puede evidenciar que el símbolo “(” se imprime como “<tkn_opening_par, fila, columna>” y que el símbolo “>=” se imprime como “<tkn_geq, fila, columna>”, entre otros. Así mismo, se debe proceder con cada uno de los tokens presentados en la tabla.

Asimismo, note que, dado que el lenguaje es basado en Javascript, sólo se contemplan los operadores y símbolos diligenciados en la tabla anterior, por lo que operadores correspondientes a operaciones bit a bit, entre otros, no se consideran para el desarrollo de esta práctica con el objetivo de delimitar un alcance adecuado para los objetivos del curso.

Además, también recuerde que Lua permite especificar únicamente el signo negativo para valores numéricos, mientras que para valores numéricos positivos Lua no soporta una especificación explícita. En este caso, el signo negativo deberá reportarse como un token adicional.

Entrada	Salida
<pre> local my_Var1 = 2 local my_Var2 = -3.145 string = "double" string' string2 = "'single' string" -- another single comment </pre>	<pre> <local,1,1> <id,my_Var1,1,7> <tkn_assign,1,15> <tkn_num,2,1,17> <local,2,1> <id,my_Var2,2,7> <tkn_assign,2,15> <tkn_minus,2,17> <tkn_num,3.145,2,18> <id,string,3,1> <tkn_assign,3,8> <tkn_str,"double" string,3,10> <id,string2,4,1> <tkn_assign,4,9> <tkn_str,'single' string,4,11> </pre>

Por último, tenga en cuenta que si en la entrada de texto se detecta una cadena como esta:

120.075.389

Se deberá seguir el **principio de la subcadena más larga**. Por consiguiente, el analizador deberá detectar un `tkn_num` con lexema 120.075; luego, detectar un `tkn_period` y, por último, un `tkn_num` con lexema 389. Esto ocurre porque el “.” podría aparecer como un token único en este lenguaje; de lo contrario, sería un error léxico. En este caso no existe un error léxico.

Errores léxicos

Cuando el analizador detecta un error léxico es necesario **abortar el análisis** y, después, se debe reportar el error por la **salida estándar** mediante un mensaje con el siguiente formato:

```
>>> Error lexico (línea: X, posicion: Y)
```

Donde **X** y **Y** son números enteros que representan el número de línea (fila) y la posición (columna) donde fue detectado el **inicio** del error léxico. Por ejemplo:

Entrada	Salida
<code>print("Valor de @: ",@)</code>	<pre><print,1,1> <tkn_opening_par,1,6> <tkn_str,Valor de @: ,1,7> <tkn_comma,1,21> >>> Error lexico (línea: 1, posicion: 22)</pre>

Note que aquí el símbolo de arroba (“@”) se considera como un error léxico, puesto que **no está definido** dentro de los tokens válidos para el lenguaje. El resto de la entrada, después del error léxico detectado, no deberá ser analizada.

A la hora de realizar el análisis léxico se deberá seguir el principio de **subcadena más larga**, incluso cuando la cadena de entrada presenta errores léxicos. Si se detecta, por ejemplo, la cadena 8.9!62834127, el analizador deberá detectar el tkn_num 8.9 y luego marcar un error léxico en el !. Recuerde que también debe tener en cuenta casos extremos, como por ejemplo:

Entrada	Salida
<code>_f 4.559<>6="1"ñvari8</code>	<pre><id,_f,1,1> <tkn_num,4.559,1,4> <tkn_less,1,9> <tkn_greater,1,10> <tkn_num,6,1,11> <tkn_assign,1,12> <tkn_str,1,1,13> >>> Error lexico (línea: 1, posicion: 16)</pre>

Observaciones finales

Los casos de prueba citados en este documento pueden no servir igualmente por el cambio de formato de los caracteres que hace el editor de texto. Podrá consultar más

ejemplos de casos de prueba en la plataforma de evaluación. Se recomienda que trabaje con los ejemplos disponibles en la plataforma de evaluación para empezar a validar que su analizador funcione de manera correcta.